



Trabajo Práctico – Callbacks, Promesas y Async/Await

Objetivo

Comprender las **tres formas principales de manejar la asincronía en JavaScript**:

1. **Callbacks**
2. **Promesas**
3. **Async/Await**

Al finalizar, vas a poder simular cómo funcionan las peticiones a un servidor (APIs) y manejar casos de éxito y error.

Contexto

En la vida real, cuando usamos JavaScript en la web, muchas veces necesitamos pedir información a un servidor (por ejemplo, usuarios, productos o noticias).

👉 Eso no ocurre inmediatamente: el servidor puede tardar 1 o 2 segundos en responder.

👉 Para simular esa espera usaremos **setTimeout**, que nos permite “fingir” una demora.

Código base

Copia este array en tu archivo main.js:

```
// Base de datos simulada (nuestros usuarios)
const users = [
  { id: 1, name: "Ana" },
  { id: 2, name: "Luis" },
  { id: 3, name: "María" }
];
```

Actividad 1: Callback

1. Creá una función `getUserById(id, callback)` que reciba:
 - un id de usuario,
 - una función callback que se ejecutará después de 1.5 segundos.
2. Dentro del `setTimeout`, buscá el usuario con `find`.
3. Si existe → devolver el usuario.
4. Si no existe → devolver un error.

👉 Ejemplo esperado:

```
getUserById(2, (error, user) => {  
  if (error) {  
    console.error("Error:", error);  
  } else {  
    console.log("Usuario encontrado:", user);  
  }  
});
```

Actividad 2: Promesas

1. Creá una función `getUserByIdPromise(id)` que retorne una **Promesa**.
2. Usá `resolve` si se encuentra el usuario y `reject` si no existe.
3. Probá la función con `.then()` y `.catch()`.

👉 Ejemplo esperado:

```
getUserByIdPromise(3)  
  .then(user => console.log("Usuario encontrado:", user))  
  .catch(error => console.error("Error:", error));
```

Actividad 3: Async/Await

1. Creá una función `async function fetchUser(id)`.
2. Usá `await` para esperar el resultado de `getUserByIdPromise(id)`.
3. Manejá el error con `try/catch`.

👉 Ejemplo esperado:

```
fetchUser(1); // debería mostrar el usuario con id 1
```

Ayuda para pensar

- **Callback** = pasás una función como argumento, y se ejecuta después de un tiempo.
- **Promesa** = un objeto que representa algo que “se resolverá en el futuro” (resolve/reject).
- **Async/Await** = forma más sencilla y “limpia” de escribir código asíncrono usando promesas.